SEGMENTO PROTECT

Universal Data Protection System (UDPS)

Protection Engine, File Processing, Privacy Transformations and Security Architecture

PUBLIC PRODUCT DOCUMENTATION
Feature availability, integrations, test results, legal references, and security parameters should be verified for the target deployment before publication.

Executive Summary

The Universal Data Protection System (UDPS), also referred to in the supplied material as the Segmento Protection Engine, is described as a privacy transformation platform for processing sensitive data in files across structured, semi-structured, columnar, document, and image formats.

The design combines file-type detection, format-specific parsing, sensitive-data recognition, format-preserving masking, one-way hashing, authenticated encryption, and controlled downloads through opaque tokens. The intended goal is to help protect personal data while preserving the structure required by downstream workflows.

Capabilities, supported formats, algorithms, and security parameters described here should be confirmed against the deployed build before being presented as guarantees.

Purpose and Scope

UDPS is intended to provide a consistent workflow for uploading a file, identifying sensitive content, selecting a protection operation, processing the file with a format-aware handler, and retrieving the result through a controlled download mechanism.

Primary operations described in the supplied material are masking, one-way hashing, authenticated encryption, and restoration/decryption for supported encrypted files. The correct operation depends on the use case: masking supports reduced-exposure data use, hashing supports deterministic comparison in appropriate designs, and encryption supports confidentiality with reversible recovery when the key material is available.

Architecture and Processing Pipeline

1. Client/upload layer: accepts a file and requested operation. The supplied architecture notes a client port of 3000; this is deployment-specific and should not be assumed for every environment.

2. File-type validation: inspects file signatures/magic bytes and MIME information to reduce reliance on user-supplied extensions. Validation should reject unsupported or malformed files safely.

3. Format-handler factory: routes the file to a format-specific parser and serializer. The supplied design describes dedicated handlers for tabular, hierarchical, columnar, document, and image formats.

4. Sensitive-data detection: combines pattern-based detection with contextual recognition. Detection results should include the detected entity type, location, confidence or review status where available, and the action applied.

5. Transformation layer: applies selected masking, hashing, or authenticated encryption. Format-aware processing should preserve required structure and types and report any unsupported constructs.

6. Opaque download vault: provides access to processed output through a high-entropy token rather than exposing filesystem paths. Tokens should be unguessable, scoped, time-limited, and invalidated according to the configured policy.

Supported File Formats

The supplied format matrix lists CSV, TSV, JSON, JSONL/NDJSON, YAML, XML, HTML, PDF, Parquet, Avro, ORC, JPEG, and PNG. This is a list of formats described in the source material; support for a given operation can vary by format and build.

Format-specific approaches described include delimited-text parsing for CSV/TSV; recursive traversal for JSON; line-by-line processing for JSONL/NDJSON; safe parsing for YAML; hierarchy-aware XML processing; DOM processing for HTML; text redaction/encryption for PDF; columnar/schema-aware processing for Parquet and ORC; schema-aware serialization for Avro; and OCR-based detection with image-region redaction for JPEG/PNG.

Before production use, validate edge cases such as malformed files, very large files, nested structures, unusual encodings, compressed data, embedded objects, scanned PDFs, transparent images, and format features not supported by the active parser.

Sensitive Data Detection

The supplied design describes a dual-pass detector combining regular-expression heuristics with contextual NLP/transformer recognition. Pattern-based rules can identify well-structured identifiers such as phone numbers or email addresses; contextual recognition can help identify sensitive values in less predictable text.

Detection quality depends on the supported entity types, language, data quality, and evaluation set. False positives and false negatives should be measured. Where incorrect masking could break a workflow or leave sensitive data exposed, provide review and validation mechanisms.

Examples of entity types in the supplied material include government identifiers, PAN, phone numbers, email addresses, names, and other configured PII patterns.

Format-Preserving Masking

Masking replaces or obscures sensitive values while retaining selected characteristics needed for a test or analytics workflow. The supplied examples include masking all but the last four digits of a 12-digit identifier, obscuring most characters in a PAN-like identifier, retaining a phone number's country code and last four digits, partially masking an email local part, and replacing name characters while retaining limited shape information.

These are illustrative patterns, not a guarantee of legal compliance or anonymisation. Retaining part of an identifier may still permit identification in context. Choose masking rules according to the data classification, intended use, and re-identification risk.

Validation should ensure that the transformed output is syntactically valid for the target format, does not accidentally expose original values elsewhere in the file, and preserves only the schema or structure that the workflow requires.

One-Way Hashing

The supplied design lists SHA-256, SHA3-256, and BLAKE3. These algorithms produce deterministic digests for the same input under the same parameters. A keyed construction such as HMAC can be appropriate where predictable hashes of low-entropy identifiers would otherwise be vulnerable to guessing.

Hashing is not encryption: the original value is not recovered by decrypting the digest. Hashing alone does not automatically anonymise personal data, especially when inputs are guessable or hashes can be linked to other datasets. Document the algorithm, any keying/salting strategy, and intended use.

Validation should cover deterministic behaviour, input encoding, canonicalisation, key handling where applicable, and compatibility across the systems that consume the resulting tokens.

Authenticated Encryption and Restoration

The supplied design describes AES-256-GCM for authenticated encryption and PBKDF2 for deriving key material from a passphrase, with a unique salt, a 96-bit IV, and a 128-bit authentication tag. The source notes 310,000 PBKDF2 iterations and a 256-bit random salt; these parameters should be verified in the active implementation and reviewed against current security guidance.

Authenticated encryption provides confidentiality and integrity when implemented correctly. Nonces/IVs must not be reused with the same key in ways prohibited by the cipher, secrets must be protected, and errors must not leak sensitive information.

The documented restore flow decrypts an encrypted file when the correct passphrase/key material is provided and rejects a file whose authentication tag fails verification. A round-trip test should compare the restored bytes with the original input, not merely compare rendered text.

Tokenised Download Vault and Temporary Data

The supplied design describes opaque, high-entropy download tokens and a 10-minute time-to-live, with internal filesystem paths withheld from clients. Tokens should be single-use if that is the configured behaviour, expire reliably, and be bound to the intended operation and authorised requester.

Temporary uploads, transformed outputs, tokens, and intermediate files should be deleted according to a verified retention policy. Ensure cleanup also handles application restarts and failed processing. Avoid logging passwords, raw PII, tokens, or sensitive file contents.

Security Gateway and REST API

The supplied API reference lists POST /api/process-file for upload and transformation, POST /api/restore-file for decryption, GET /api/download/:token for protected retrieval, GET /api/formats for format metadata, GET /api/health for health status, POST /oauth/token for token issuance, and GET /.well-known/jwks.json for public key discovery.

The supplied design also references OAuth 2.0/OIDC, RS256-signed JWTs, scope-based access, path traversal protections using resolved and validated root paths, short-lived download tokens, and avoidance of password logging.

Public API documentation should describe authentication requirements, request limits, accepted MIME types, error responses, token expiry, rate limits, and the exact behaviour of each endpoint. Health endpoints should avoid revealing sensitive internal details.

Quality Assurance and Testing

The supplied project notes report 20 test suites and 234 automated tests for UDPS. Treat these as source-provided figures and publish them only after verifying the latest CI/test run and the code version to which the numbers apply.

Test scenarios described in the source include sensitive-entity recognition, format-preserving serialisation across supported formats, hashing consistency, AES-GCM round-trip restoration, authentication-tag tampering, token expiry, path traversal prevention, and OAuth/JWKS/scope checks.

Additional recommended tests include malformed and oversized uploads, parser timeouts, memory limits, partial writes, interrupted downloads, concurrent requests, expired or reused tokens, key rotation, cleanup after crashes, and safe error handling.

Example Workflow

Example: an operator uploads a CSV containing customer contact information and chooses masking. The system validates the file signature and format, parses records, detects configured sensitive fields, applies the selected masking rules, serialises the output, and stores the result behind a short-lived opaque download token.

The operator downloads the transformed file through the token endpoint and checks that expected fields are masked while the CSV structure remains usable. For encryption, a separate test encrypts a file and restores it using the correct passphrase, then verifies byte-for-byte equality. A modified ciphertext should fail authentication and should not produce a successful restore.

Deployment and Website Presentation Notes

Only advertise the formats and operations verified in the published build. If a feature depends on optional libraries, external AI services, OCR binaries, or a Python bridge, identify the dependency and its availability in the deployment.

Do not publish credentials, keys, real personal data, internal filesystem paths, or sensitive operational configuration. Avoid claims such as 'military-grade', 'zero risk', 'fully anonymised', or 'unbreakable'. State the actual algorithm and the controls implemented.

This document is technical product documentation and is not a security certification or legal opinion. Security parameters and deployment claims should be reviewed by the responsible engineering and security teams.

Document status and review

Prepared from the project-provided technical specification. Confirm current implementation, supported configuration, test results, legal references, and security parameters with the responsible teams before publication.